

Super Simple Sorting Code - Easy to Remember! 🚀

🎯 The EASIEST way to remember sorting algorithms!

1 Selection Sort - "Find Smallest, Swap"

🧠 Remember: Find MIN, put it FIRST

```
cpp
// Selection Sort - Super Simple!
for(int i = 0; i < n-1; i++) {
    int min = i;

    for(int j = i+1; j < n; j++) {
        if(arr[j] < arr[min])
            min = j;
    }

    swap(arr[i], arr[min]);
}
```

Memory Trick: "Select the smallest, swap to front"

2 Insertion Sort - "Pick & Insert"

🧠 Remember: Like sorting CARDS in your hand

```
cpp
```

```
// Insertion Sort - Like Cards!
```

```
for(int i = 1; i < n; i++) {  
    int key = arr[i];  
    int j = i - 1;  
  
    while(j >= 0 && arr[j] > key) {  
        arr[j+1] = arr[j];  
        j--;  
    }  
  
    arr[j+1] = key;  
}
```

Memory Trick: "Pick a card, shift bigger ones right, insert"

3 Bubble Sort - "Compare & Swap Neighbors"

 **Remember: Bubbles go UP (big numbers go right)**

```
cpp
```

```
// Bubble Sort - Easiest!
```

```
for(int i = 0; i < n-1; i++) {  
    for(int j = 0; j < n-i-1; j++) {  
        if(arr[j] > arr[j+1])  
            swap(arr[j], arr[j+1]);  
    }  
}
```

Memory Trick: "If left > right, swap them"

4 Merge Sort - "Split, Sort, Merge"

 **Remember: Break it down, build it back**

```
cpp
```

// Merge Sort - Divide & Conquer

```
void mergeSort(int arr[], int l, int r) {  
    if(l < r) {  
        int m = (l + r) / 2;  
  
        mergeSort(arr, l, m);    // Left half  
        mergeSort(arr, m+1, r);  // Right half  
        merge(arr, l, m, r);     // Combine  
    }  
}
```

// Merge function (just copy this)

```
void merge(int arr[], int l, int m, int r) {  
    int n1 = m - l + 1;  
    int n2 = r - m;  
    int L[n1], R[n2];  
  
    for(int i = 0; i < n1; i++) L[i] = arr[l+i];  
    for(int i = 0; i < n2; i++) R[i] = arr[m+1+i];  
  
    int i = 0, j = 0, k = l;  
    while(i < n1 && j < n2) {  
        if(L[i] <= R[j]) arr[k++] = L[i++];  
        else arr[k++] = R[j++];  
    }  
    while(i < n1) arr[k++] = L[i++];  
    while(j < n2) arr[k++] = R[j++];  
}
```

Memory Trick: "Split until 1 element, merge back sorted"

5 Quick Sort - "Pick Pivot, Partition"

 **Remember:** Small left, Big right, Repeat

cpp

// Quick Sort - Fast!

```
void quickSort(int arr[], int low, int high) {  
    if(low < high) {  
        int pivot = arr[high];  
        int i = low - 1;  
  
        for(int j = low; j < high; j++) {  
            if(arr[j] < pivot) {  
                i++;  
                swap(arr[i], arr[j]);  
            }  
        }  
        swap(arr[i+1], arr[high]);  
  
        int pi = i + 1;  
        quickSort(arr, low, pi-1); // Left side  
        quickSort(arr, pi+1, high); // Right side  
    }  
}
```

Memory Trick: "Pivot = boss. Small left, big right, repeat"

SUPER QUICK CHEAT SHEET

Selection Sort (3 lines core):

```
cpp  
  
min = i  
find smaller → update min  
swap(arr[i], arr[min])
```

Insertion Sort (3 lines core):

```
cpp  
  
key = arr[i]  
shift bigger ones right  
insert key
```

Bubble Sort (2 lines core):

```
cpp  
  
if(arr[j] > arr[j+1])  
    swap
```

Merge Sort (1 line core):

```
cpp  
  
split → sort left → sort right → merge
```

Quick Sort (1 line core):

```
cpp  
  
partition around pivot → sort left → sort right
```

💡 Remember Just THIS Pattern!

All sorting has same structure:

```
cpp  
  
for(int i = 0; i < n; i++) {  
    // Do something with arr[i]  
}
```

The difference?

- **Selection:** Find min in remaining
 - **Insertion:** Insert current in sorted part
 - **Bubble:** Swap with neighbor if wrong
 - **Merge:** Recursion + merge
 - **Quick:** Recursion + partition
-

Visual Memory Aid

Selection: [? ? ? ?] → Find MIN → [✓ ? ? ?]

Insertion: [✓ ✓ ? KEY ?] → Shift → [✓ ✓ KEY ✓ ?]

Bubble: [5 2] → Swap → [2 5]

Merge: [4 2] + [3 1] → [1 2 3 4]

Quick: [3 1 5 2]₄ → [3 1 2] 4 [5]

Exam Quick Write

If you forget everything, write Bubble Sort (easiest):

cpp

```
for(int i = 0; i < n; i++)  
    for(int j = 0; j < n-1; j++)  
        if(arr[j] > arr[j+1])  
            swap(arr[j], arr[j+1]);
```

That's it! 4 lines!

Practice This Pattern

Write 10 times each day:

Day 1: Bubble Sort (easiest)

Day 2: Selection Sort

Day 3: Insertion Sort

Day 4: Quick Sort

Day 5: Merge Sort

By Day 5, you'll remember all! 💪

Final Tip

Don't memorize line by line! Memorize the IDEA:

- Selection = Find smallest
- Insertion = Insert like cards

- Bubble = Swap neighbors
- Merge = Split & merge
- Quick = Partition & recurse

The code will come naturally! ✨

🔥 ONE Sentence Summary

- **Selection:** Find min, swap front
- **Insertion:** Pick one, shift, insert
- **Bubble:** Compare neighbors, swap
- **Merge:** Split half, sort, combine
- **Quick:** Pick pivot, split, recurse

Remember these 5 sentences = Remember all sorting! 🎉